Data-Driven Targeting of Likely Electric Vehicle Buyers

An Ensemble Approach with Gradient-Boosted and Neural Network Models

Technical report · Artur Markosyan · September 2026

Top 10%	0.9455	0.9465	~70
final rank 307 of ~3,300 teams	ROC AUC on the final hold-out set	ROC AUC in 10-fold cross-validation	candidate models evaluated

Summary

The goal was to rank people by how likely they are to buy an electric vehicle, so that incentives and marketing can be focused on the most likely buyers. The model is an ensemble of gradient-boosted decision trees and neural networks trained on 668,665 labelled records with 13 inputs (age, income, commute, charging access, environmental concern, subsidy, range anxiety and others). Every modelling decision was made on out-of-fold predictions from 10-fold cross-validation, never on the leaderboard.

The work was benchmarked in an open machine-learning competition with about 3,300 participating teams. The final solution scored a ROC AUC of 0.94546 on the hidden final test set (rank 307, top 10%), up from 0.94509 for the starting single-notebook model. A notable finding: submissions tuned to the public leaderboard scored higher there (up to 0.94661) but lower on the hidden final set than the cross-validated ensemble, which confirmed the validation-first approach.

What the report covers

- How the problem and evaluation were set up (sections 1–2)
- Features and models, and how the ensemble combines them (sections 3–5)
- Every idea that was tested, with the result and the decision to accept or reject it (section 6)
- Results, lessons and limitations (sections 7–8); credits and reproducibility (appendix)

1. Problem and data

Task. Binary classification: for each person, predict the probability that they will buy an electric vehicle (*Will_Buy_EV*, yes/no). Only 17.5% of training records are buyers, so the classes are imbalanced.

Data. 668,665 labelled training records and 286,571 unlabelled test records. A smaller 10,000-row reference dataset from the same domain was also available and was tested as an extra source of features.

Group	Input columns
Demographics	Age, Gender, Annual income (USD)
Mobility	Daily commute (km), Number of cars owned, Current car type, City type
Charging access	Charging stations near home, Charging stations near work, Home charging possible
Attitude & policy	Environmental concern level (1–5), Subsidy available, Range anxiety level

Metric. ROC AUC: the probability that a randomly chosen buyer is ranked above a randomly chosen non-buyer (0.5 = random, 1.0 = perfect). AUC depends only on the *order* of predictions, which is used heavily later: models are combined by their ranks, and post-processing only moves rows up or down the ranking.

Evaluation. Test predictions were scored on two hidden splits: a public 20% shown during the competition, and a final 80% revealed only at the end, which determines the rank. The public split is small enough (about 57,000 rows) that differences of 0.0001 AUC are within noise.

2. Validation strategy

Every model was evaluated with **10-fold stratified cross-validation** (fixed seed, identical folds across all of my models). For each fold, the model trains on 90% of the rows and predicts the held-out 10%. Stitching the held-out predictions together gives an **out-of-fold (OOF)** prediction for every training row from a model that never saw that row's label. The OOF AUC over 668,665 rows is a far more stable estimate than the 57,000-row public split, and the OOF predictions are also the inputs for choosing ensemble weights.

Safeguards

- **Leakage-safe encodings.** Any feature computed from labels (target encodings, window rates) is cross-fitted: training rows get values from an inner 5-fold split, so no row ever sees its own label.
- **Honest ensemble estimates.** Ensemble weights were fitted on one half of the rows and scored on the other half (4 random splits, both directions), so the reported ensemble gain is not optimistic.
- **Nested validation for post-processing.** Rules and corrections were accepted only if they improved AUC on every fold of an independent 5-fold split.
- **Leakage screening of inputs.** One external candidate reported an OOF AUC of exactly 1.000, which is impossible for an honest out-of-fold file; it was excluded.
- **One decision rule.** Nothing was accepted because of a public-leaderboard score.

3. Feature engineering

Tree models already handle raw columns well, so most of the gain comes from encodings that expose the *exact* value of income and commute. Analysis showed that individual income values carry real, value-specific purchase effects of about ±4 percentage points beyond what a smooth income curve explains, and the features below are different ways of letting a model use that information without overfitting.

Feature family	What it captures	How it is computed
Raw inputs	The 13 original columns	Categoricals as native categories
Digit features	Fine-grained structure of income and commute values	Individual digits (ones, tens, …) of income and commute
Frequency encoding	How common each income / commute value is	Count of the value in train + test
Target encoding	Purchase rate for each value of a column	Smoothed mean label per value, cross-fitted; several smoothing strengths (m = 5, 20, 100, 200)
Multi-resolution keys	Purchase rate at coarser income / commute levels	Target encoding of income rounded to 10 / 100 / 1,000 and commute to 1 / 5 km
Window encodings	Purchase rate among people with *similar* income or commute	Smoothed rate within ±2 … ±1,000 USD and ±0.1 … ±10 km, via prefix sums; cross-fitted
Interaction encodings	Combined effects (e.g. subsidy × range anxiety, income band × subsidy)	Target encoding of 12 column combinations

Feature ideas tested and rejected

Before building full models, candidate feature sets were screened with a fast 5-fold LightGBM against a fixed baseline (AUC 0.94586). Every idea below lowered the score and was rejected. The clearest lesson: removing columns that look irrelevant (age, gender, number of cars, car type) cost 0.00047, because they still carry signal the trees use.

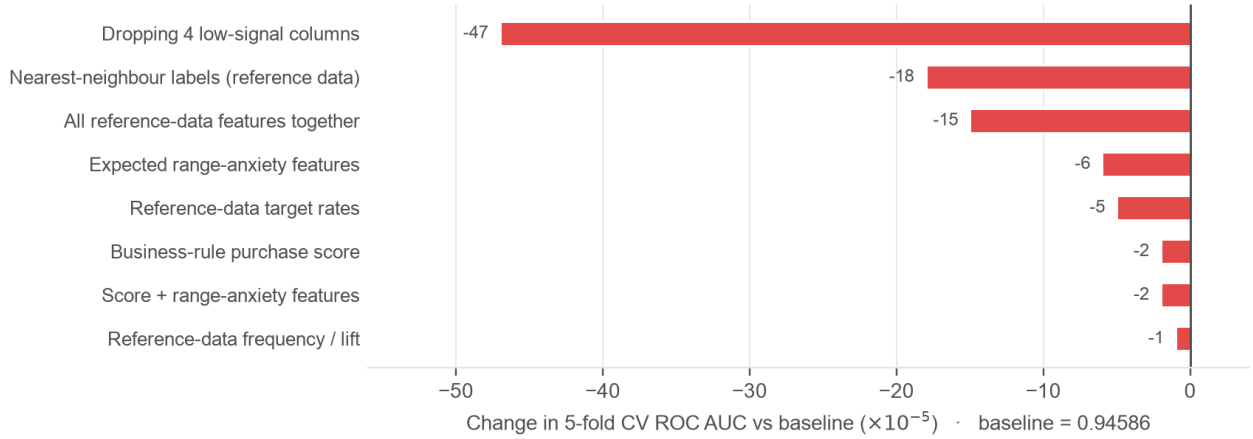


Figure 1. Fast screening of feature ideas (5-fold LightGBM). All ideas lowered validation AUC relative to the baseline and were rejected.

4. Models

Two model families were used. **Gradient-boosted decision trees** (LightGBM, XGBoost, CatBoost) build thousands of small trees in sequence, each correcting the errors of the previous ones; they are the strongest single models for tabular data. **Neural networks** (multi-layer perceptrons, RealMLP, TabM) learn smooth functions of the inputs; they are slightly weaker alone but make *different* errors, which is what an ensemble needs.

Models I trained

Model	What is distinctive	OOF AUC	Outcome
LightGBM (starting notebook, 3 seeds)	Nested target encodings, digits, frequencies	0.94611	Pool member
CatBoost (starting notebook)	Native categorical handling	0.94602	Pool member
LightGBM + window encodings	Similar-income / similar-commute purchase rates	0.94612	Pool member
LightGBM + interaction encodings	12 column-combination encodings	0.94606	Pool member
Depth-2 LightGBM	Near-additive model (pairwise interactions only)	0.94603	Pool member
MLP + periodic embeddings (3 seeds)	Learned sin/cos embeddings of income & commute	0.94593	Pool member
MLP + interaction encodings (3 seeds)	Neural view of the combination features	0.94596	Pool member
RealMLP (re-implemented, 3 seeds)	Modern tabular network, run on a local GPU	≈0.9460 / seed	Pool member
XGBoost, hierarchical encodings (2 new seeds)	Re-runs of the strongest external model on cloud GPU	0.94640 (3-seed avg)	**Top ensemble member**
XGBoost, window encodings (1 new seed)	Re-run of an external model on cloud GPU	0.94628 (2-seed avg)	Pool member
LightGBM with pseudo-labels	Test rows added with ensemble probabilities as soft labels	0.94601	Rejected
TabM (cloud GPU)	Parameter-efficient ensemble network	0.94015	Rejected

External candidate models

Other competition participants published out-of-fold and test predictions for their models. Because out-of-fold predictions are honest by construction, these could be evaluated and weighted with exactly the same procedure as my own models. About 60 external candidates from 15 sources were added to the pool (individual OOF AUC 0.938–0.9463), including XGBoost and LightGBM models with different encoding schemes, several RealMLP variants, CatBoost models, and simpler linear, spline (GAM) and random-forest models. Sources are credited in the appendix. Files were aligned by row id and checked for leakage before use.

5. How the ensemble works

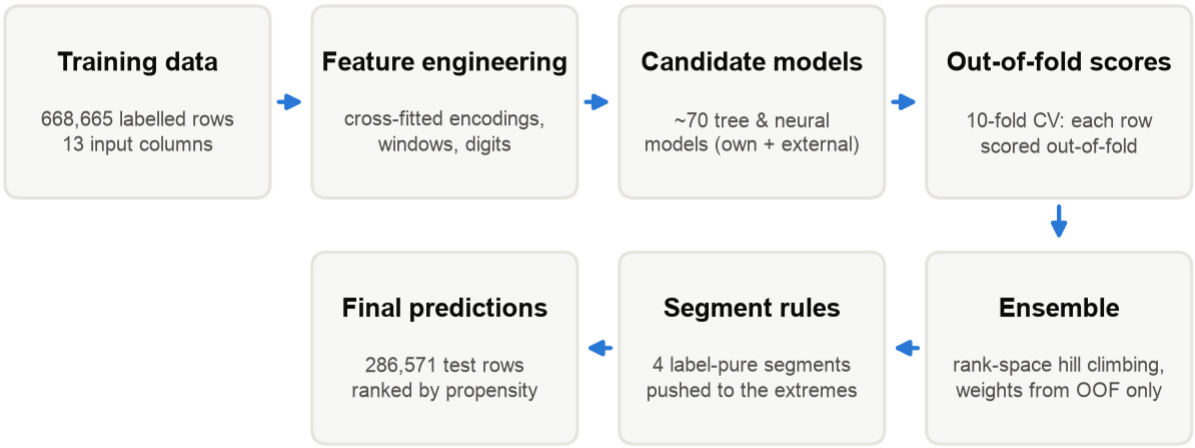


Figure 2. End-to-end pipeline. All selection decisions use out-of-fold predictions; the test set is only scored at the end.

Rank space. Models output probabilities on different scales (some even raw scores). Because AUC depends only on order, every prediction is converted to its rank (0–1) before combining, which makes all models directly comparable.

Hill climbing (Caruana ensemble selection). Start from the best single model; repeatedly add (with replacement) whichever model most improves the OOF AUC of the running average; stop when no addition helps. Weights are therefore multiples of 1/8 and the method automatically ignores redundant models. With ~70 highly correlated candidates (pairwise correlation 0.99+), this simple, low-parameter method generalised better than every more flexible alternative tested:

Combiner	OOF AUC	Held-out gain over best single model	Decision
Hill climbing (rank space)	0.94650	+0.00008 (±0.00000)	**Used**
Logistic-regression stacking	0.94649	+0.00005 (±0.00002)	Rejected
Non-negative logistic stacking	0.94641–0.94643	+0.00008–0.00009 (earlier pool)	Rejected
LightGBM level-2 stacker	0.94634	below logistic stacking	Rejected
Equal-weight average of all models	—	−0.00007	Rejected

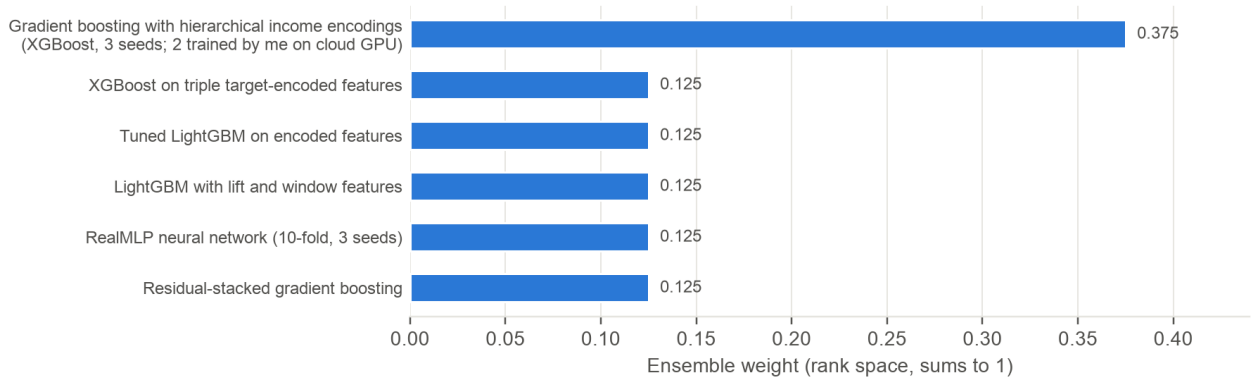


Figure 3. Final ensemble: six members selected by hill climbing from ~70 candidates. Selection is purely data-driven: my own models were competitive individually but were superseded by external models carrying the same information more strongly. The top member combines the original run with two seeds I trained on cloud GPU.

Segment rules (post-processing)

Four segments of the input space have a constant label across all 668,665 training rows. Tree models almost learn them, but averaging many models smooths the edges, so a few rows inside a "never buys" segment were ranked above 70% of the data. Enforcing the segments moves those rows to the very top or bottom of the ranking while keeping the order inside each segment.

Segment	Training rows	Buy rate	Test rows
Income ≥ 170,537 USD	393	100%	156
Income 31,004–41,970 USD	1,257	0%	494
Daily commute ≥ 83 km	186	0%	81
Income exactly 30,000, no subsidy, and (concern level 1 or medium/high anxiety)	7,157	0%	3,002

Validated on an independent 5-fold split, the rules improved AUC on **5 of 5 folds** (mean +0.0000056). The gain is small but consistent, so the rules were accepted.

6. Experiment log: what was accepted and what was rejected

Every idea was judged by the same rule: does it improve out-of-fold AUC (and, for post-processing, on every fold)? Changes are relative to the configuration at the time of the test.

Accepted

Idea	Effect	Why it worked
Rank-space hill-climbing ensemble	+0.00045 OOF over the starting notebook (0.94597 → 0.94642)	Combines models that make different errors
Expanding the candidate pool with external OOF predictions	+0.00006 OOF (0.94644 → 0.94650 with seeds)	New feature views and model types add diversity
Within-fold rank normalisation for neural nets	+0.00008 on the neural-net member	Neural-net calibration drifted between folds
Seed averaging of the top members (cloud GPU)	+0.00009 on the top member, +0.00002 on the ensemble	Reduces model variance where the weight is largest
Label-pure segment rules	+0.0000056 OOF, 5 of 5 folds	Restores hard edges that averaging smooths away

Rejected

Idea	Result	Reason
Business-rule purchase score as a feature	−0.00002 CV	Trees already learn the same rule
Expected range-anxiety features	−0.00006 CV	Redundant with existing columns
Reference-dataset frequency, lift and target-rate features	−0.00001 to −0.00005 CV	Reference data too small to add signal
Nearest-neighbour labels from the reference dataset	−0.00018 CV	Noisy; adds variance
Dropping 4 low-signal columns	−0.00047 CV	Those columns still carry signal
Linking each record to a specific reference row	≤ +0.00003 (measured by others)	Too small to justify the complexity
Hypothesis: labels follow a tree fitted on the reference data	Not supported by the data	Same segments are not pure there (47%, 10%, 15% buy rates)
Pseudo-labelling (soft labels on test rows)	0.94601 vs 0.94612 without	Worse on 8 of 10 folds

Idea	Result	Reason
TabM network (cloud GPU)	0.94015; lowers the ensemble at every weight	Too weak in this setting
Residual target encoding on the ensemble	0.000000 on all folds	Ensemble already captures per-value effects
Level-2 LightGBM / non-negative logistic stacking	0.94634 / 0.94641–0.94643	Overfit with highly correlated inputs
Blending in files tuned to the public leaderboard	Public 0.94661, final 0.94544	Higher public score, lower final score than the CV ensemble

7. Results

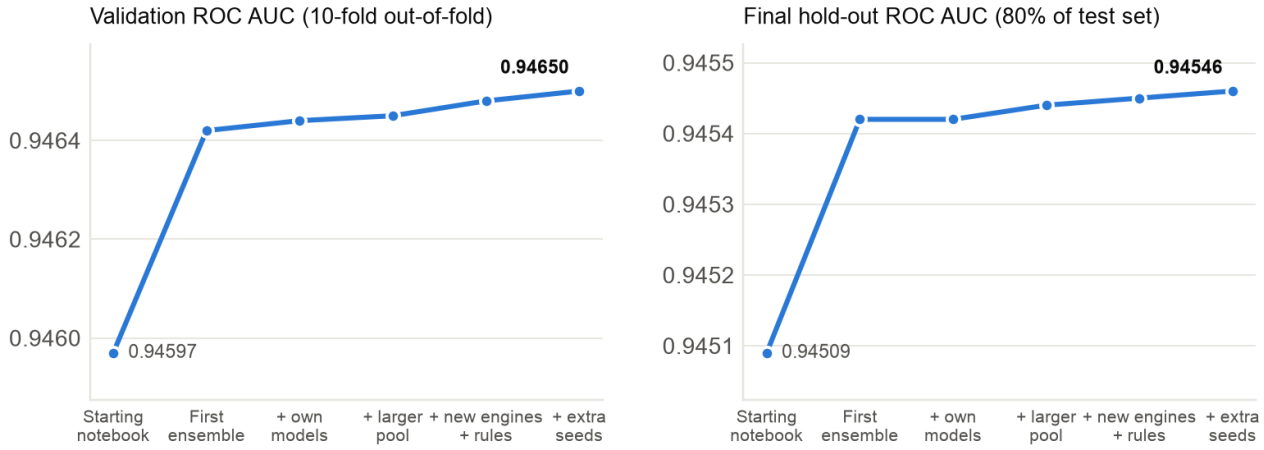


Figure 4. Validation AUC (left) and final hold-out AUC (right) across the main stages. Both improve together, which shows the cross-validation tracked the hidden test set.

Submission	OOF AUC	Public (20%)	Final (80%)
Starting notebook (single ensemble)	0.94597	0.94609	0.94509
First rank-space ensemble	0.94642	0.94647	0.94542
Larger candidate pool	0.94645	0.94649	0.94544
+ new external models + segment rules	0.94648	0.94649	0.94545
Final ensemble (+ seed averaging)	**0.94650**	0.94647	**0.94546**
For comparison: leaderboard-tuned public file	n/a	0.94661	0.94544
Competition winner	n/a	n/a	0.94602

Final rank: 307 of about 3,300 teams (top 10%). The final ensemble improved the hidden-test AUC by 0.00037 over the starting notebook. On the public split its score stayed around 0.94647–0.94649 while validation kept improving; the final split confirmed the validation ranking, not the public one.

8. Lessons and limitations

- **Validate on the largest honest sample available.** 668,665 out-of-fold predictions were a better guide than a 57,000-row public split; files tuned to the public split lost their advantage on the final set.
- **Diversity beats individual strength in ensembles.** Models with a different view of the data earned weight even when slightly weaker; strong but near-duplicate models earned none.
- **Simple combiners generalise best with correlated inputs.** Hill climbing beat logistic, non-negative and tree-based stackers.

- **Gains were small by the end.** After the first ensemble, further work added +0.00008 OOF in total. The winning solution (0.94602) was 0.0006 ahead, which suggests a structural insight about the data rather than more models.
- **Limitations.** The final ensemble relies largely on externally published models; my contribution is the validation and selection pipeline, my own candidate models, the extra seed runs and the segment rules. Compute was limited to a laptop plus free cloud GPUs (two concurrent sessions), which ruled out large hyper-parameter searches.

Appendix A. Reproducibility

File	Purpose
quick_checks.py	Fast 5-fold screening of feature ideas against a fixed baseline
view_model.py	10-fold LightGBM / XGBoost / CatBoost on configurable feature views (windows, interactions, depth)
mlp_model.py	10-fold PyTorch MLP with optional periodic embeddings
realmlp_local.py	Local re-implementation run of a public RealMLP architecture
pool.py	Loads, aligns (by row id) and rank-normalises all OOF / test prediction pairs
blend.py	Hill-climbing and logistic stacking with honest half-split estimates
stack_l2.py, blend_nn.py	Level-2 LightGBM and non-negative logistic stackers (rejected)
rules.py, finish_blend.py	Segment rules and final file assembly
Cloud-GPU notebooks	Notebook forks run on cloud GPUs (TabM, extra seeds) and the orchestration script

Appendix B. Credits

External out-of-fold predictions and published model code were used from the following competition participants (usernames): megayak (feature-view libraries), najiama (OOF hub), lucifer19 (fusion components, segment rules write-up), heuljax (hierarchical-encoding XGBoost), blamerx and medvax (window-encoding XGBoost), yekenot (RealMLP), chinzorigtganbat (LightGBM, CatBoost, TabM), mizushimatoshihiko (LightGBM), tamerlanomralinov (fingerprint models), legtarrr (residual stack), dariushafshar (model library), marcmaldonado (row-level analysis), aryankaisth (HistGBM), evgendvorkin and kirill0212 (XGBoost / LightGBM baselines), udaken10 and stephentarter (CatBoost / HistGBM), and Chris Deotte (exploratory analysis of the reference data). The segment rules were found by the community and independently verified here.